

CENTRAL DE MISSÕES DA LIGA DA JUSTIÇA

Central -LJ

Documentação Técnica do Sistema

Full-Stack · Event-Driven · Arquitetura Hexagonal

PROJETO

Projeto Integrador de Módulo

DISCIPLINAS

Design · Web · Mensageria
· Qualidade

ENTREGA

N1 + N2

BANCA

03 / 06 / 2026

Spring Boot 3.4 · React 19 · Kafka KRaft · PostgreSQL 16 · JWT · SSE

Sumário

0

Sumário

1	Visão Geral e Contextualização	1
1.1	Problema resolvido	1
1.2	Escopo funcional	1
1.3	Fluxo principal	1
2	Arquitetura do Sistema	1
2.1	Arquitetura Hexagonal	1
3	Diagramas de Arquitetura	2
3.1	C4 – Contexto	2
3.2	C4 – Contêineres	3
3.3	Classes – Domínio	3
3.4	Pipeline Kafka	5
3.5	Entidade-Relacionamento	6
4	Modelo de Domínio	6
4.1	Entidades principais	6
4.2	Máquina de estado – MissionStatus	6
5	Princípios SOLID e Padrões de Projeto	6
5.1	Princípios SOLID	6
5.2	Padrões de Projeto	6
6	Mensageria com Apache Kafka	7
6.1	Topologia	7
6.2	Perfis	7
6.3	Contrato de evento	7
7	Tempo Real, Segurança e Frontend	7
7.1	SSE (tempo real)	7
7.2	JWT e papéis (N2)	7
7.3	Rotas frontend	7
8	Qualidade, Testes e RNFs	7
8.1	Suítes de teste (25 total)	7
8.2	Requisitos não-funcionais	7
9	Guia de Execução	8
9.1	Credenciais demo	8

1 Visão Geral e Contextualização

A **Central-LJ** simula a central de operações da **Liga da Justiça**: plataforma full-stack orientada a eventos (N1), com autenticação JWT, papéis distintos e área do herói (N2). O processamento assíncrono usa **Apache Kafka**; a UI recebe atualizações via **SSE**.

♦ **Escopo de entrega:** N1 (eventos e workflow Kafka) + N2 (auth, herói, atribuição, arquitetura hexagonal). A API registra a missão, publica no Kafka e retorna imediatamente; o workflow roda em background.

5

Entidades de domínio (N1/N2)

20+

Endpoints REST

25

Testes automatizados

Problema resolvido

Centrais de operações não podem bloquear o operador. A arquitetura orientada a eventos desacopla **registro e processamento**: confirmação imediata na API, priorização e escalonamento em background.

Escopo funcional

Módulo	Funcionalidades	Perfil
Missões	CRUD, workflow Kafka, atribuição herói/equipe, histórico	ADMIN/OP
Heróis e Equipes	Cadastro, disponibilidade, vínculos	ADMIN/OP
Área do Herói	Missões designadas, timeline, perfil	HERO
Autenticação	Login JWT, seed demo, controle de papéis	Público
Dashboard	KPIs, volume por status, missões recentes	ADMIN

Fluxo principal

Et.	Ação	Resultado
1	POST /api/missions	Requisição recebida
2	Persistência JPA	RECEBIDA + histórico
3	After-commit → Kafka	missions.created
4	Consumer + Strategy	Transições de status
5	SSE mission-update	Dashboard atualizado

2 Arquitetura do Sistema

Arquitetura Hexagonal

O backend isola o domínio de Spring, JPA, Kafka e HTTP via **Ports & Adapters**.

Camada	Conteúdo
Adapters IN	6 controllers REST · 2 consumers Kafka · JWT e policies
Application	10+ use cases · 10+ services · Strategy · commands/views
Domain	5 entidades · enums — POJOs puros (escopo N1/N2)
Adapters OUT	5 JPA adapters · produtor Kafka · SSE

♦ **Benefício:** cada adaptador é substituível. Testes mockam ports sem subir Kafka, servlet ou PostgreSQL.

Estrutura de pacotes

```
br.edu.central.centrallj/
├── adapter/in/web|messaging/      ← REST, DTOs, JWT, consumers
├── adapter/out/persistence|messaging|realtime/
├── application/port/in|out/       ← Use cases e ports
├── application/service/workflow/ ← Strategy de prioridade
└── domain/                      ← Entidades e enums puros
```

3 Diagramas de Arquitetura

Diagramas formais com legenda numerada. Figuras compactas maximizam legibilidade sem desperdiçar páginas.

C4 — Contexto

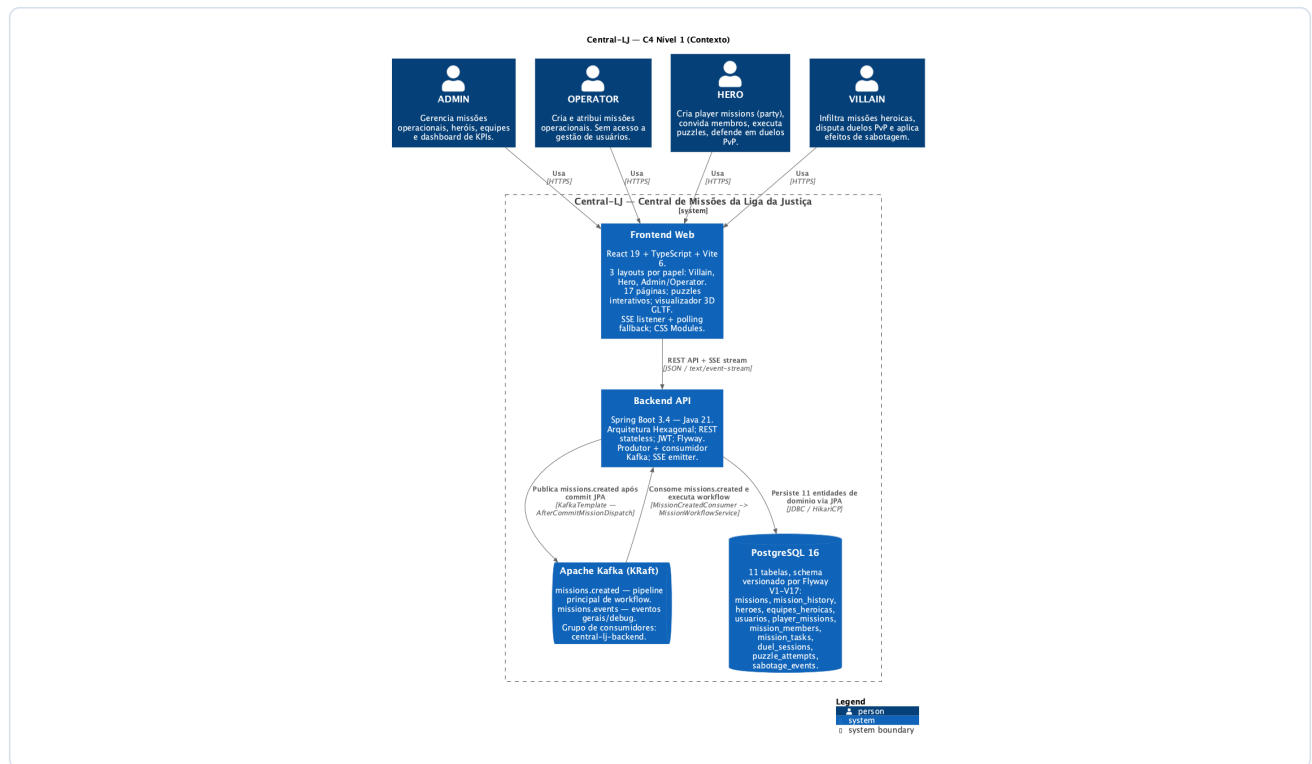


Fig. 1: C4 Nível 1 — atores ADMIN, OPERATOR e HERO; sistemas Kafka e PostgreSQL.

Elemento	Tipo	Relação
ADMIN / OPERATOR	Pessoa	Coordena missões, dashboard, elenco heroico
HERO	Pessoa	Acompanha missões designadas em /heroi/area
SPA React	Software	REST + SSE · layouts admin e herói
Spring Boot API	Software	Regras, JWT, Kafka, JPA
Kafka / PostgreSQL	Software	Eventos + persistência relacional

C4 – Contêineres

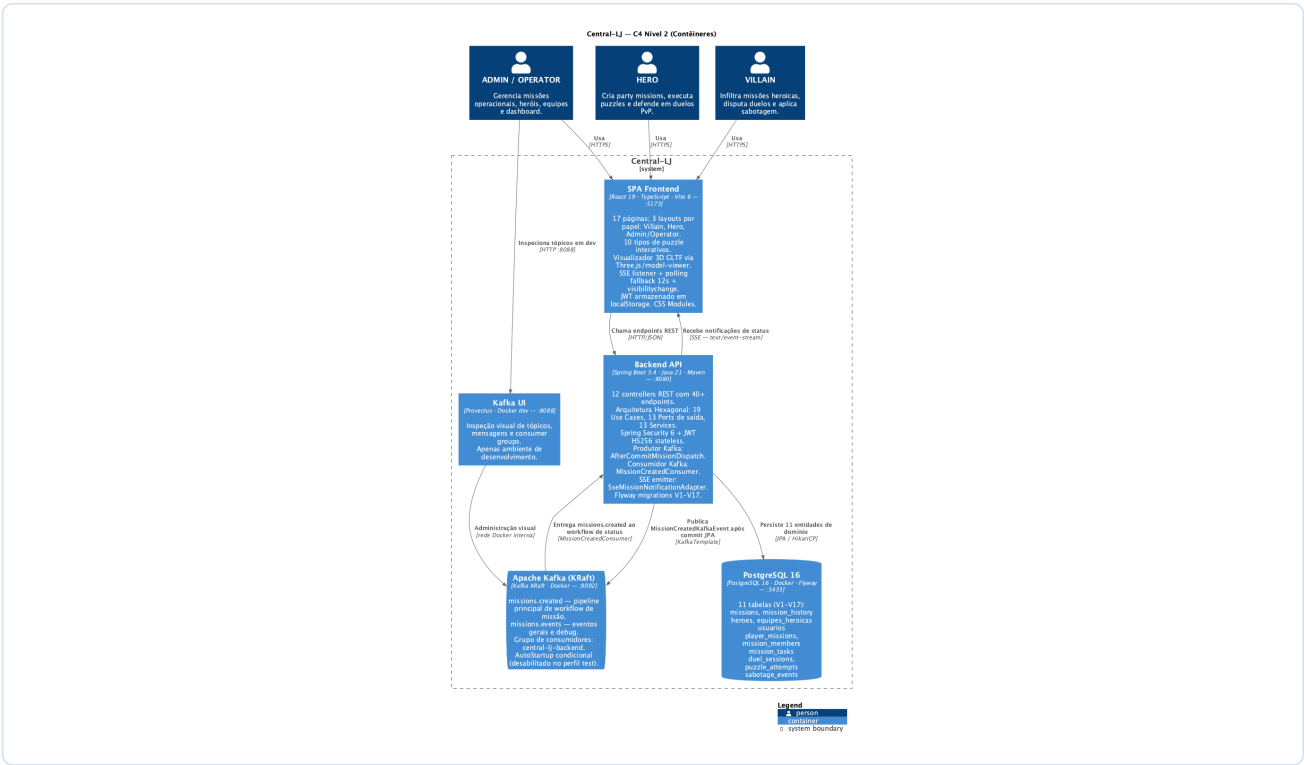


Fig. 2: C4 Nível 2 — SPA, API, Kafka KRaft, PostgreSQL e Kafka UI (:8088).

Contêiner	Stack	Papel
SPA	React 19 · Vite 6	Painel admin + área do herói · SSE + polling
API	Spring Boot 3.4	REST · Security · Flyway · Kafka in/out
Kafka	KRaft :9092	Workflow assíncrono missions.created
PostgreSQL	:5433	Missões, heróis, equipes, usuários, histórico

Classes — Domínio

Central-LJ — Modelo de Domínio

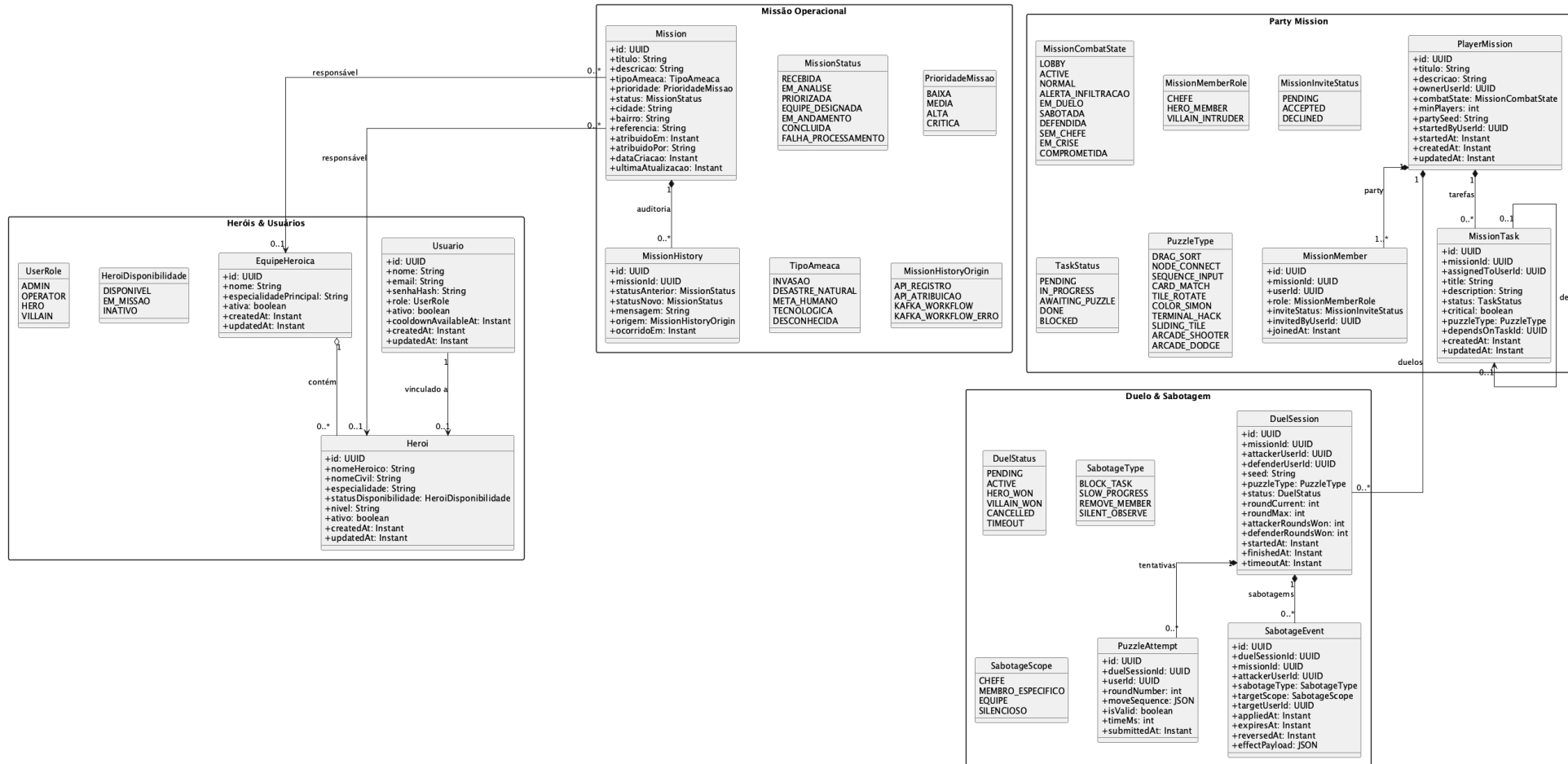


Fig. 3: Entidades de domínio N1/N2 — Missão, Herói, Equipe, Usuário e histórico auditável.

Pipeline Kafka

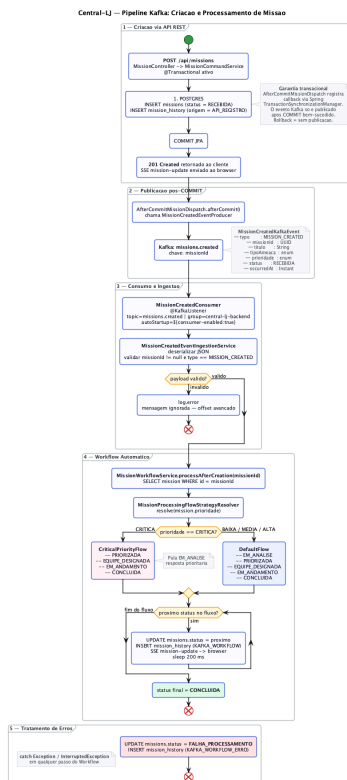


Fig. 4: Fluxo missions.created: API → after-commit → consumer → workflow → SSE.

Componente	Responsabilidade
MissionCommandService	Persiste RECEBIDA; dispara evento interno
AfterCommitMissionDispatch	Publica só após commit JPA
MissionCreatedEventFactory	Invariantes: eventId, missionId, type
MissionCreatedEventProducer	JSON → tópico missions.created
MissionWorkflowService	Strategy por prioridade + histórico
SseMissionNotificationAdapter	Push mission-update

Entidade-Relacionamento

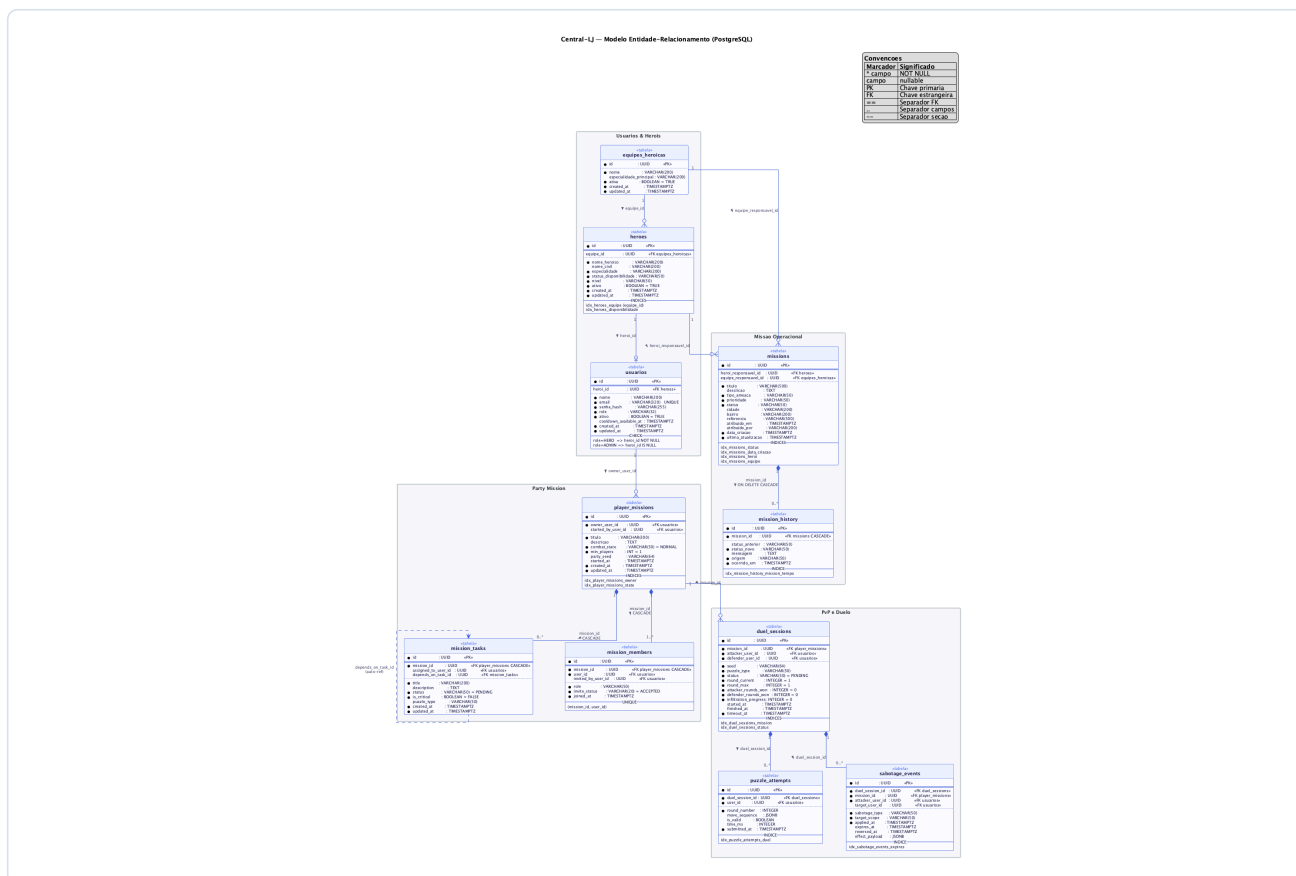


Fig. 5: Modelo relacional PostgreSQL 16 – núcleo N1/N2 (missões, elenco, auth, histórico).

4

Modelo de Domínio

Entidades principais

Entidade	Atributos-chave	Papel
Mission	prioridade, status, local, heroId, equipeId	Missão operacional
MissionHistory	statusAnterior/Novo, origem, ocorridoEm	Auditoria imutável
Heroi	nome, disponibilidade, equipeId	Elenco em campo
EquipeHeroica	nome, descricao	Agrupamento de heróis
Usuario	email, senhaHash, role, heroId	Auth JWT e papéis globais

Máquina de estado — MissionStatus

RECEBIDA → EM_ANALISE → PRIORIZADA → EQUIPE_DESIGNADA → EM_ANDAMENTO → CONCLUIDA
Missões **CRÍTICA**: fast-track via CriticalPriorityMissionProcessingFlowStrategy.
Falha no pipeline: FALHA_PROCESSAMENTO.

5

Princípios SOLID e Padrões de Projeto

Princípios SOLID

P	Aplicação
S	Command / Query / Workflow / History separados
O	Novas strategies sem alterar consumer
L	Strategies substituíveis no resolver

Padrões de Projeto

Padrão	Onde
Strategy	Workflow por prioridade
Factory	MissionCreatedEventFactory
Repository	Ports JPA desacoplados
DTO / Record	DTOs imutáveis na camada web

P	Aplicação	Padrão	Onde
I	Use cases granulares por operação REST	Observer	AfterCommitMissionDispatch
D	Services → ports, nunca JPA/Kafka	Chain	Security: CORS → JWT → AuthZ

```
interface MissionProcessingFlowStrategy {  
    List<MissionStatus> getSteps();  
    boolean supports(PrioridadeMissao p);  
}  
// Default → 5 etapas | CriticalPriority → fast-track
```

6 Mensageria com Apache Kafka

Núcleo do requisito **não-CRUD**: reação a fatos de domínio imutáveis. Ver Fig. 4.

Topologia

Tópico	Papel
missions.created	Workflow principal
missions.events	Debug / testes

Perfis

Perfil	Config
local/prod	Kafka :9092 · PG :5433
test	Consumer off · H2 in-memory

Contrato de evento

```
{ "eventId": "uuid", "missionId": "uuid", "type": "MISSION_CREATED", "occurredAt": "ISO-8601" }
```

7 Tempo Real, Segurança e Frontend

SSE (tempo real)

Unidirecional servidor→cliente.
point GET /api/missions/stream,
ter SseMissionNotificationAdapter e
useMissionUpdates com polling de fallback (12 s).

End-
adap-
hook

JWT e papéis (N2)

Papel	Acesso
ADMIN	Governança completa — missões, dashbo- ard, elenco
OPERATOR	Criar/listar missões, assign hero/team
HERO	/api/me/missions — missões designadas a ele

Rotas frontend

Rota	Página	Perfil
/	Dashboard KPIs	ADMIN
/missoes	Lista e detalhe + timeline	ADMIN/OP
/herois, /equipes	Elenco heroico	ADMIN/OP
/heroi/area	Área do herói	HERO
/login	Autenticação JWT	Público

8 Qualidade, Testes e RNFs

Suítes de teste (25 total)

Suíte	N
MissionApiIntegrationTest	9
HeroEquipeAssignmentApiIntegrationTest	4
AuthApiIntegrationTest	3
MissionWorkflowIntegrationTest	3
Unitários (Ingestion, Strategy, etc.)	6

Requisitos não-funcionais

Cat.	Atendimento
Desempenho	API < 100 ms — Kafka async
Escalabilidade	Consumer group horizontal
Segurança	JWT + @PreAuthorize
Testabilidade	H2 + consumer off em CI
Auditoria	mission_history imutável

◆ **BUILD SUCCESS** — JaCoCo: backend/target/site/jacoco/index.html

9 Guia de Execução

Pré-requisitos: Java 21 · Node 20+ · Docker Desktop

```
cd infra && docker compose up -d
cd backend && ./mvnw spring-boot:run # :8080
cd frontend && npm i && npm run dev # :5173
cd backend && ./mvnw test
```

Health: GET /api/health → {"status":"ok"}

Credenciais demo

Papel	E-mail	Senha
ADMIN	coordenacao@central-lj.demo	Admin@demo2026
HERO	heroi.demo@central-lj.demo	Hero@demo2026

Variável	Descrição
JWT_SECRET	Segredo HS256 (≥ 32 chars)
KAFKA_BOOTSTRAP_SERVERS	Broker Kafka
DATABASE_URL	JDBC PostgreSQL

Central-LJ — Central de Missões da Liga da Justiça

PUC Goiás · ADS 4º Período · Semestre 2026/1 · Entrega N1 + N2